

MULTI-LINE INSURANCE POLICY AND CLAIMS MANAGEMENT SYSTEM

Salesforce Implementation Project Documentation

Team Leader: Mohamed Sharif N

Team Members: Vishal Manikandan R, Vijaymanikandan S, Thirumalaivasan R, George
Daniel Raj. A

College: Arasu Engineering College

Registered Mail ID: mhdsharif135@gmail.com

Team ID: SWTID-2026-5200

Platform: Salesforce Developer Edition Org (Lightning Experience)

Submission Date: 3rd October 2026

Contents

1. Project Overview
 2. Objectives
 3. Problem Statement and Requirements
 4. Project Planning
 5. Implementation (Data Model, Validation, Flows, Apex, LWC, Approval, Security)
 6. Testing
 7. Challenges Faced and Fixes
 8. Limitations and Future Scope
 9. Conclusion
- Appendix A: Apex and LWC Source Code

1. Project Overview

The Multi-Line Insurance Policy and Claims Management System is a Salesforce CRM application for an insurance business that sells Vehicle, Property and Life insurance. In many insurance offices quoting, policy issuance and claim handling are still done through separate manual steps, which makes them slow and inconsistent. This project brings policies, claims, approvals and reporting into one Salesforce org so that agents, adjusters and managers work from one shared and current set of records.

The application is built on two custom objects, **Policy** and **Claim**, together with the standard Contact and User objects. Policy-specific Record Types and Field Sets let one object hold three different insurance products. A guided Screen Flow (*AutoQuotingFlow*) captures an auto quote and creates a draft policy, a validation rule checks the VIN, and an Apex class (*PremiumCalculator*) works out the premium. When a claim is created, a record-triggered Flow routes it to the Auto, Property or Life queue. Claims above 50,000 go through a two-step approval process (Senior Adjuster, then Manager). Adjusters see their own claims on a Lightning Web Component dashboard, and access is controlled with state-based sharing rules and three Permission Sets.

All configuration and code was created and tested by our team inside a Salesforce org, working in Lightning Experience.

2. Objectives

- Hold customer, policy and claim data for Vehicle, Property and Life insurance in one structured system.
- Standardise and speed up policy quoting through a guided Screen Flow.
- Calculate policy premiums automatically with Apex instead of by hand.
- Reject invalid vehicle data, such as a VIN that is not exactly 17 characters.
- Route every new claim automatically to the correct claims queue based on policy type.
- Send high-value claims through a controlled, multi-level approval process.
- Give claims adjusters a focused dashboard of their assigned claims.
- Limit what each kind of user can see and do using sharing rules and Permission Sets.

3. Problem Statement and Requirements

3.1 Problem Statement

Insurance operations deal with several product lines at once, and each line needs different information. Without a central system, quoting needs several manual steps, claims are reviewed and assigned by hand, high-value claims have no formal escalation route, and nobody has one complete view of customer, policy and claim data. The result is slow processing, inconsistent data and unhappy customers. Ten customer problem statements (PS-1 to PS-10) were captured during ideation; they come from the Insurance Operations Manager, Insurance Agent, Claims Manager, Claims Adjuster, Senior Adjuster, Customer and Insurance Management roles.

3.2 Users and Their Needs

User	What they need from the system
Insurance Agent	To create policies, run a guided auto quote and see the calculated premium.
Claims Adjuster	To see only the claims assigned to them, with amount, policy type, policy holder, status and days open.
Senior Adjuster / Claims Manager	To review high-value claims, approve or reject them with comments, and run reports.
Administrator	To configure objects, flows, security and monitor the automation.
Customer	To get a policy issued and a claim handled quickly.

3.3 Functional Requirements

ID	Requirement	Delivered by
FR-01/02	Create and manage Vehicle, Property and Life policies using separate Record Types.	Policy__c with Auto, Property, Life Record Types
FR-03	Capture product-specific data (VIN, Model Year, Square Footage, Year Built, Beneficiary).	Custom fields and Field Sets
FR-04/05	Guided auto quoting that creates a draft policy.	Screen Flow AutoQuotingFlow
FR-06	Calculate premium with Apex.	PremiumCalculator (invocable Apex)
FR-07	VIN must contain exactly 17 characters.	Validation rule VIN_Must_Be_17_Characters
FR-08/09	Create claims linked to policies, with Accident, Property and Life Record Types.	Claim__c with Record Types
FR-10	Route claims to the right queue by policy type.	Record-triggered Flow on Claim
FR-11/12/13	Detect high-value claims; multi-level approval with Approve/Reject and comments.	Approval process, Submission Automation Flow, Claim Approver Screen Flow
FR-14/15/16	Claims dashboard with key metrics and All/Auto/Property/Life filter.	claimsDashboardLwc, claimTileLwc, ClaimsAdjusterController
FR-17	Automatic updates to related Policy and Claim records.	Record-triggered Flows
FR-18/19	State-based sharing and role-specific access.	Sharing rules and three Permission Sets

ID	Requirement	Delivered by
FR-20	Reporting and visibility of customer, policy and claim data.	Salesforce reports, dashboards and LWC

3.4 Non-Functional Requirements

- **Usability:** simple guided Screen Flows that minimise clicks for agents.
- **Scalability and extensibility:** three insurance lines today, with room for more products without a new structure for each.
- **Data integrity:** validation keeps policy and claim data accurate.
- **Security:** access by responsibility, using state-based sharing and Permission Sets.
- **Maintainability and reliability:** modular Flows, Apex, LWC, Record Types and Field Sets that run consistently.

3.5 Technology Stack

Layer	Technology used
Platform	Salesforce, Lightning Experience
Data model	Custom objects Policy__c and Claim__c; standard Contact and User; Record Types, Field Sets, custom fields
Declarative automation	Screen Flow, Record-Triggered Flows, Validation Rule, Approval Process, Quick Action
Programmatic	Apex (PremiumCalculator, ClaimsAdjusterController), Apex test class
Interface and analytics	Lightning Web Components (claimsDashboardLwc, claimTileLwc), reports and dashboards
Security	Permission Sets and state-based sharing rules
Tools	Setup, Object Manager, Flow Builder, Developer Console, Lightning Studio extension

3.6 How Data Moves Through the System

Step	What happens	Result
1	Agent starts the Auto Quoting Screen Flow and enters policy holder, start date, state, VIN and model year.	Basic and vehicle details captured.
2	Validation checks that the VIN has 17 characters.	Invalid VINs are rejected.
3	The flow creates a draft Policy__c with the Auto Record Type.	Draft policy exists.
4	PremiumCalculator runs and the flow updates the Premium field.	Premium stored on the policy.
5	A Claim__c is created and linked to a policy.	Claim record exists.
6	Record-triggered Flow reads the policy type and sets the claim owner to the Auto, Property or Life queue.	Claim routed.
7	If Claim Amount is above 50,000 and status is New, the claim is submitted for approval.	Senior Adjuster, then Manager review.
8	Approver chooses Approve or Reject with comments; status field is updated.	Claim Approved or Rejected.

Step	What happens	Result
9	Adjusters open the LWC dashboard.	Own claims shown with days open.

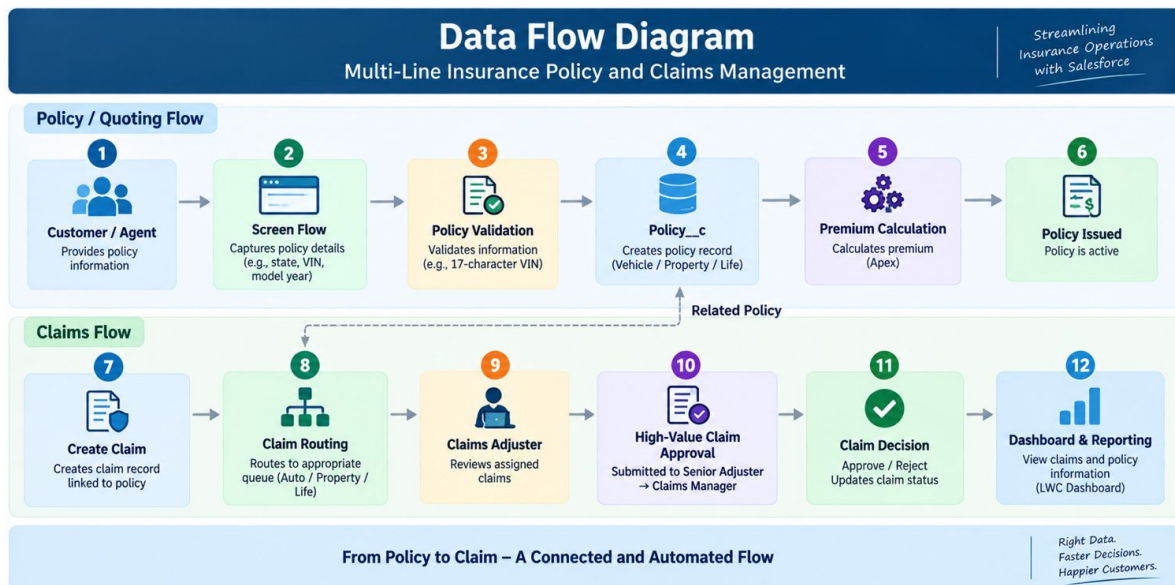


Figure 1: Data flow of the Multi-Line Insurance system

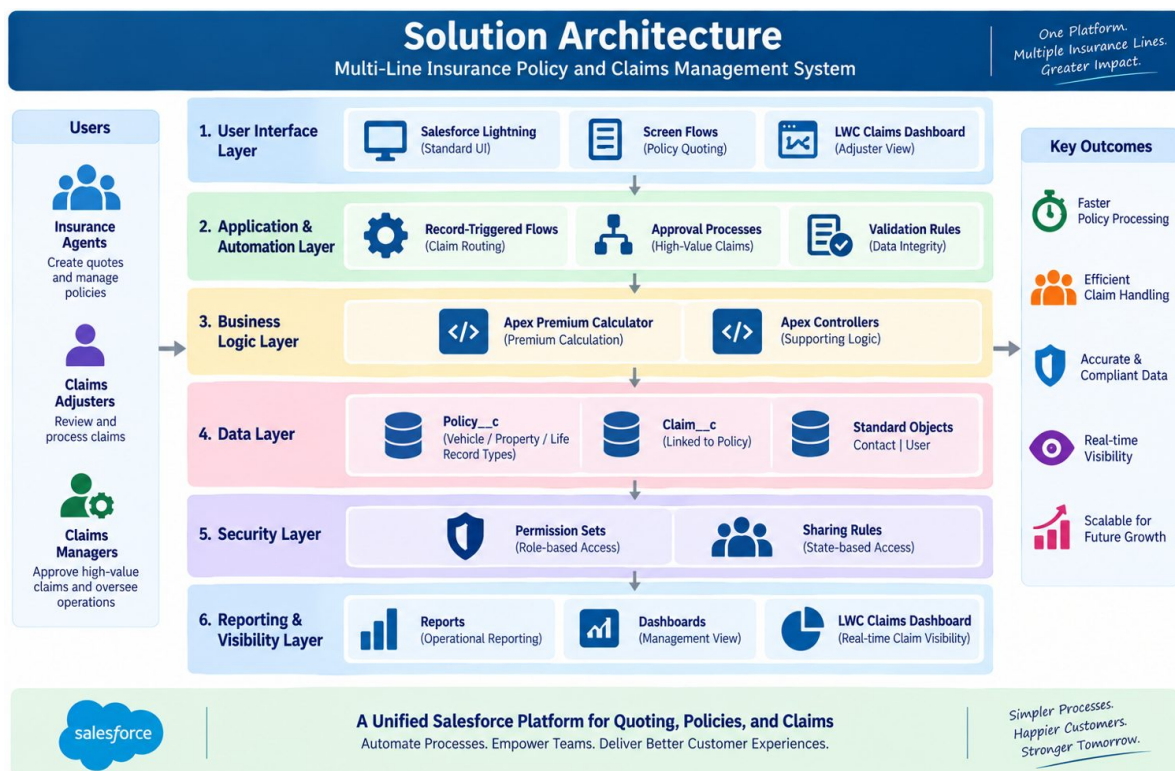


Figure 2: Layered solution architecture

4. Project Planning

The project followed an Agile approach: epics were broken into user stories, each story was given story points, and the work was delivered in four sprints of 20 story points each.

4.1 Product Backlog

Sprint	Epic	Story	User story / task	Pts	Priority
1	Data Modeling	US-01	Create Policy and Claim objects to store insurance data systematically.	5	High
1	Data Modeling	US-02	Configure Vehicle, Property and Life Record Types on Policy.	3	High
1	Data Modeling	US-03	Configure Field Sets and product-specific fields.	5	High
1	Quoting	US-04	Guided Auto Quoting Screen Flow.	8	High
2	Validation	US-05	VIN validation during vehicle quoting.	3	High
2	Apex	US-06	PremiumCalculator Apex to calculate premiums.	8	High
2	Automation	US-07	Integrate premium calculation into the quoting Flow.	5	High
2	Claims Automation	US-08	Route claims by policy type to the right queue.	8	High
3	Apex	US-09	ClaimsAdjusterController to retrieve assigned claims.	8	High
3	LWC Dashboard	US-10	Claims dashboard and reusable claim tile.	8	High
4	Approval	US-11	High-value claim approval process and approver Flow.	8	High
4	Security and Testing	US-12	Tests, sharing rules and Permission Sets.	8	High

4.2 Sprint Schedule

Sprint	Total points	Duration	Start date	End date (planned)
Sprint-1	20	1 day	29 Sep 2026	29 Sep 2026
Sprint-2	20	1 day	30 Sep 2026	30 Sep 2026
Sprint-3	20	1 day	01 Oct 2026	01 Oct 2026
Sprint-4	20	2 days	02 Oct 2026	03 Oct 2026

Velocity is 20 story points per sprint, so the 80-point backlog is finished in four sprints. Testing and documentation were completed alongside Sprint-4, and the project was submitted on 3 October 2026.

5. Implementation

5.1 Org Setup

Every object, field, flow, class and component described in this document was created in a Salesforce org through the Setup menu, the Developer Console and the Lightning Studio extension, working in Lightning Experience.

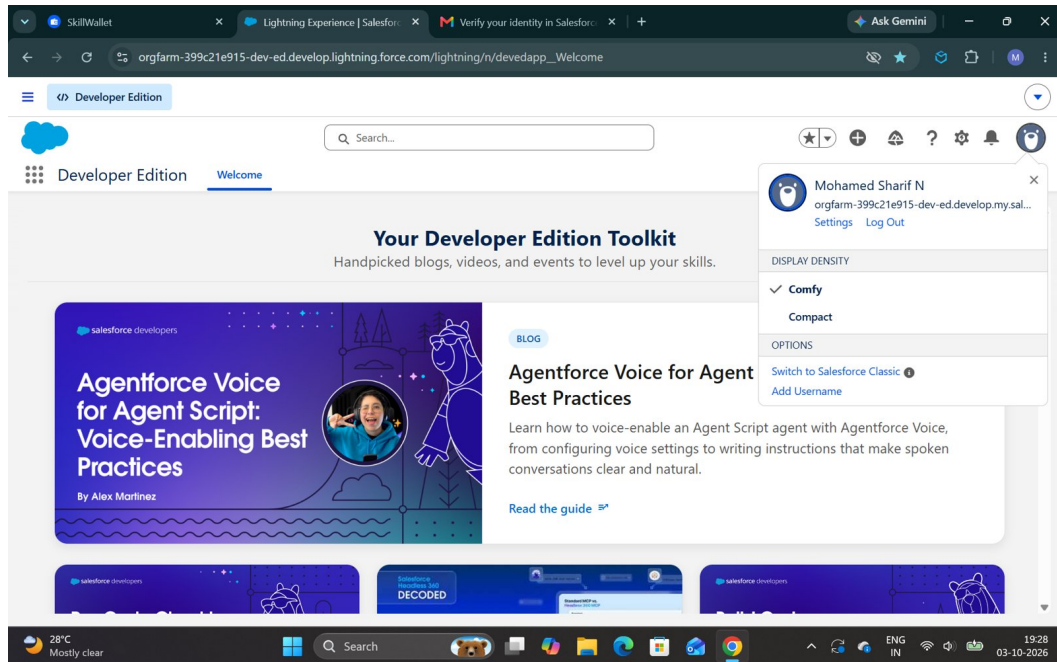


Figure 3: Developer Edition org home page with the logged-in user

5.2 Data Model

The data model has two custom objects. **Policy** (Policy__c) uses an auto-number record name in the format P-{0000}. **Claim** (Claim__c) uses an auto-number record name in the format C-{0000}. Both objects have Reports, Activities and Field History tracking turned on, are deployed, and allow search. A claim looks up to its policy, and a policy looks up to a Contact who is the policy holder.

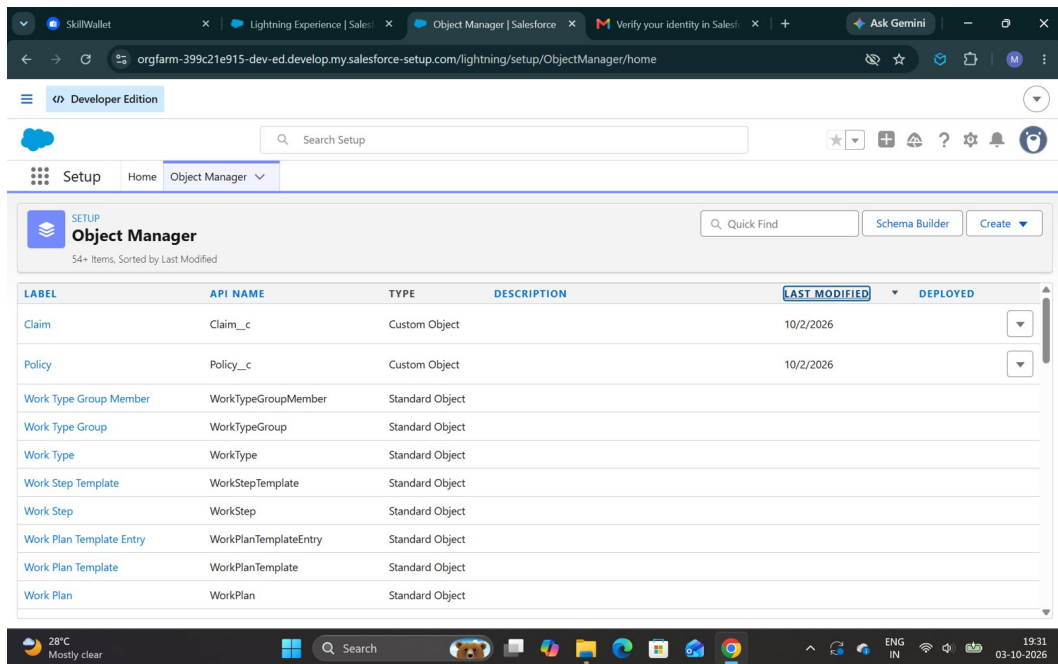


Figure 4: Object Manager showing the Policy and Claim custom objects

Object (API name)	Purpose	Fields we created
Policy (Policy__c)	One record per insurance policy of any line.	Customer (lookup Contact), Policy Start Date, Premium (Currency 16,2), Policy State (picklist), VIN (Text 20), Model Year (Text 5), Square Footage, Year Built, Beneficiary Name, Policy Term Months
Claim (Claim__c)	One record per claim raised against a policy.	Policy (lookup), Adjuster (lookup User), Claim Amount (Currency 16,2), Date of Loss (Date/Time), Description (Long Text), Approval Status (picklist: New, Submitted for Approval, Approved, Rejected), Policy Account Holder State (Text 50, copied from the policy by a Flow)

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Beneficiary Name	Beneficiary_Name__c	Text(50)		
Created By	CreatedById	Lookup(User)		
Customer	Customer__c	Lookup(Contact)		✓
Last Modified By	LastModifiedById	Lookup(User)		
Model Year	Model_Year__c	Text(5)		
Owner	OwnerId	Lookup(User,Group)		✓
Policy Number	Name	Auto Number		✓

Figure 5: Policy fields (1 of 3)

SETUP > OBJECT MANAGER			
Policy			
Details	Fields & Relationships		
Fields & Relationships	15 Items, Sorted by Field Label		
Page Layouts	Q, Quick Find		
Lightning Record Pages	New Deleted Fields Field Dependencies Set History Tracking		
Buttons, Links, and Actions			
Compact Layouts			
Field Sets			
Object Limits			
Record Types			
Related Lookup Filters			
Search Layouts			
	Policy Start Date	Policy_Start_Date__c	Date
	Policy State	Policy_State__c	Picklist
	Policy Term Months	Policy_Term_Months__c	Number(18, 0)
	Premium	Premium__c	Currency(16, 2)
	Record Type	RecordTypeId	Record Type
	Square Footage	Square_Footage__c	Number(18, 0)
	VIN	VIN__c	Text(20)

Figure 6: Policy fields (2 of 3)

SETUP > OBJECT MANAGER			
Policy			
Details	Fields & Relationships		
Fields & Relationships	15 Items, Sorted by Field Label		
Page Layouts	Q, Quick Find		
Lightning Record Pages	New Deleted Fields Field Dependencies Set History Tracking		
Buttons, Links, and Actions			
Compact Layouts			
Field Sets			
Object Limits			
Record Types			
Related Lookup Filters			
Search Layouts			
List View Button Layout			
	Policy Start Date	Policy_Start_Date__c	Date
	Policy State	Policy_State__c	Picklist
	Policy Term Months	Policy_Term_Months__c	Number(18, 0)
	Premium	Premium__c	Currency(16, 2)
	Record Type	RecordTypeId	Record Type
	Square Footage	Square_Footage__c	Number(18, 0)
	VIN	VIN__c	Text(20)
	Year Built	Year_Built__c	Text(5)

Figure 7: Policy fields (3 of 3)

SETUP > OBJECT MANAGER			
Claim			
Page Layouts	Fields & Relationships		
Lightning Record Pages	12 Items, Sorted by Field Label		
Buttons, Links, and Actions	Q, Quick Find		
Compact Layouts	New Deleted Fields Field Dependencies Set History Tracking		
Field Sets			
Object Limits			
Record Types			
Related Lookup Filters			
Search Layouts			
List View Button Layout			
Restriction Rules			
Scoping Rules			
Object Access			
	FIELD LABEL	FIELD NAME	DATA TYPE
	Adjuster	Adjuster__c	Lookup(User)
	Approval Status	Approval_Status__c	Picklist
	Claim Amount	Claim_Amount__c	Currency(16, 2)
	Claim Number	Name	Auto Number
	Created By	CreatedById	Lookup(User)
	Date of Loss	Date_of_Loss__c	Date/Time
	Description	Description__c	Long Text Area(32768)

Figure 8: Claim fields (1 of 2)

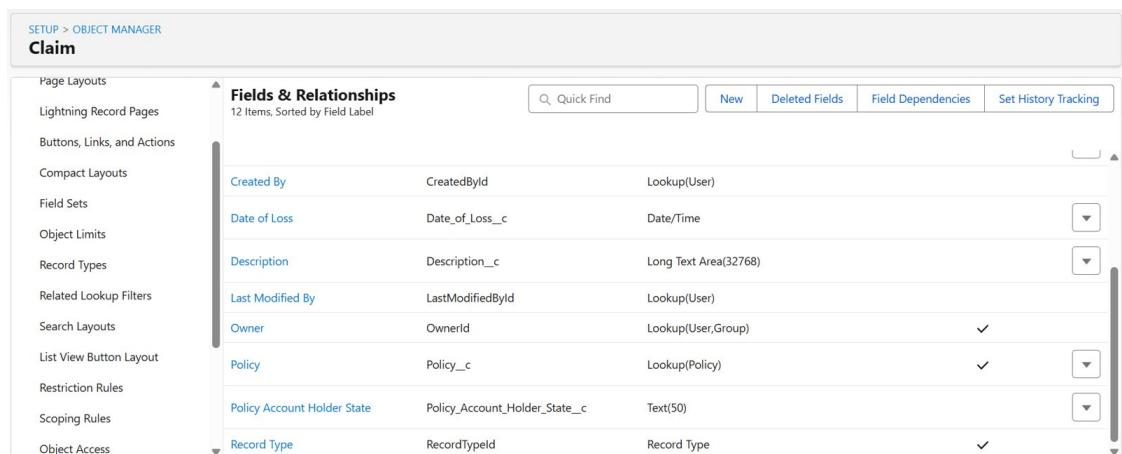


Figure 9: Claim fields (2 of 2)

Record Types

Policy has three Record Types, **Auto**, **Property** and **Life**. Claim has three Record Types, **Accident**, **Property** and **Life**. Each is active and available to all profiles. This lets one object serve three products.

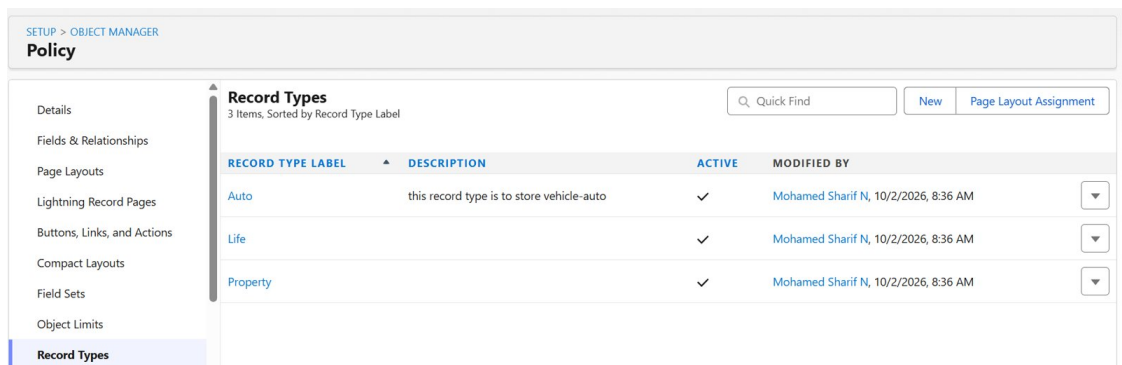


Figure 10: Record Types on the Policy object: Auto, Life and Property

Field Sets

Field Sets group the fields that belong to each product line, so each Record Type shows only the relevant data.

Field Set	Fields
Vehicle	VIN, Model Year
Property	Square Footage, Year Built
Life	Beneficiary Name, Policy Term Months

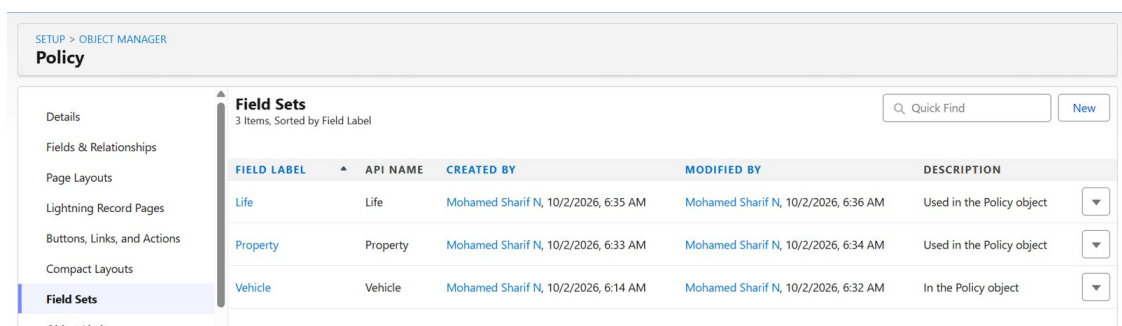


Figure 11: Field Sets on the Policy object

SETUP > OBJECT MANAGER
Policy

Vehicle Fields

Save Cancel Undo Redo Field Set Properties

Quick Find Policy Name

Beneficiary Name	Deleted	Last Referenced Date	Policy Number	Premium	System Modstamp
Created By	Last Activity Date	Last Viewed Date	Policy Start Date	Record ID	VIN
Created Date	Last Modified By	Model Year	Policy State	Record Type	Year Built
Customer	Last Modified Date	Owner	Policy Term Months	Square Footage	

Drag any of the fields above into one of the lists below.

Available for the Field Set

Drag and drop the fields you want administrators to have available to add to the Field Set.

In the Field Set

VIN
Model Year

Figure 12: Vehicle Field Set

Setup Home Object Manager

SETUP > OBJECT MANAGER
Policy

Property Fields

Save Cancel Undo Redo Field Set Properties

Quick Find Policy Name

Beneficiary Name	Deleted	Last Referenced Date	Policy Number	Premium	System Modstamp
Created By	Last Activity Date	Last Viewed Date	Policy Start Date	Record ID	VIN
Created Date	Last Modified By	Model Year	Policy State	Record Type	Year Built
Customer	Last Modified Date	Owner	Policy Term Months	Square Footage	

Drag any of the fields above into one of the lists below.

Available for the Field Set

Drag and drop the fields you want administrators to have available to add to the Field Set.

In the Field Set

Square Footage
Year Built

Figure 13: Property Field Set

Setup Home Object Manager

SETUP > OBJECT MANAGER
Policy

Life Fields

Save Cancel Undo Redo Field Set Properties

Quick Find Policy Name

Beneficiary Name	Deleted	Last Referenced Date	Policy Number	Premium	System Modstamp
Created By	Last Activity Date	Last Viewed Date	Policy Start Date	Record ID	VIN
Created Date	Last Modified By	Model Year	Policy State	Record Type	Year Built
Customer	Last Modified Date	Owner	Policy Term Months	Square Footage	

Drag any of the fields above into one of the lists below.

Available for the Field Set

Drag and drop the fields you want administrators to have available to add to the Field Set.

In the Field Set

Beneficiary Name
Policy Term Months

Figure 14: Life Field Set

5.3 Validation Rule for VIN

Vehicle policies must carry a valid VIN, so the rule **VIN_Must_Be_17_Characters** blocks the save when an Auto policy has a VIN that is not exactly 17 characters long. The error appears beside the VIN field.

Property	Value
Rule name	VIN_Must_Be_17_Characters (Active)
Object	Policy

Property	Value
Error condition formula	AND(RecordType.DeveloperName = "Auto", LEN(VIN__c) <> 17)
Error message	The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.
Error location	VIN field

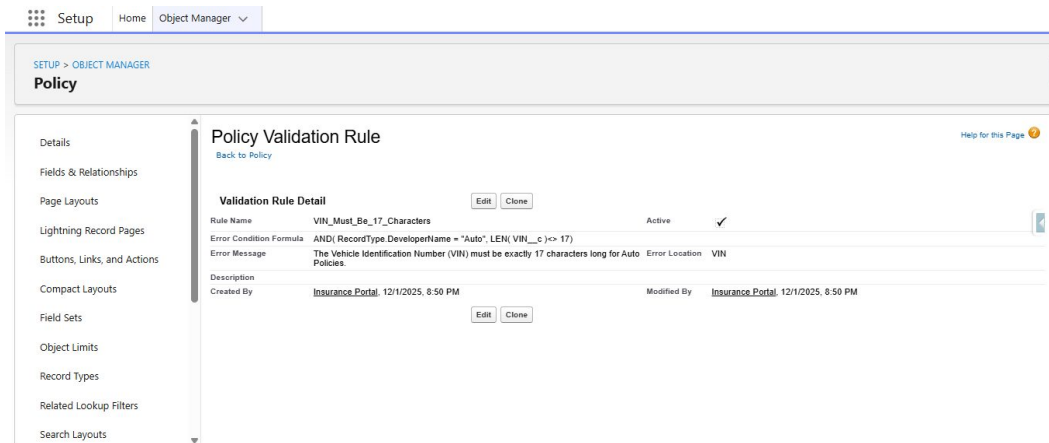


Figure 15: Validation rule VIN_Must_Be_17_Characters on the Policy object

5.4 Auto Quoting Screen Flow

AutoQuotingFlow is a Screen Flow that guides the agent through a quote. It runs these elements in order:

- **Get Vehicle RT ID** (Get Records): finds the Auto Record Type of Policy__c.
- **New Auto Policy Quote - Basic Information** (Screen): Policy Holder lookup (Customer__c), Policy Start Date and Policy State picklist.
- **Vehicle-Specific Details** (Screen): VIN and Model Year text inputs.
- **Create Draft Policy** (Create Records): stores the entered values with the Auto Record Type and saves the new Id in varPolicyId.
- **Calculate Premium Action** (Action): passes varPolicyId to PremiumCalculator and stores the result in varCalculatedPremium.
- **Update Policy With Premium** (Update Records): writes varCalculatedPremium to the Premium field of the new policy.

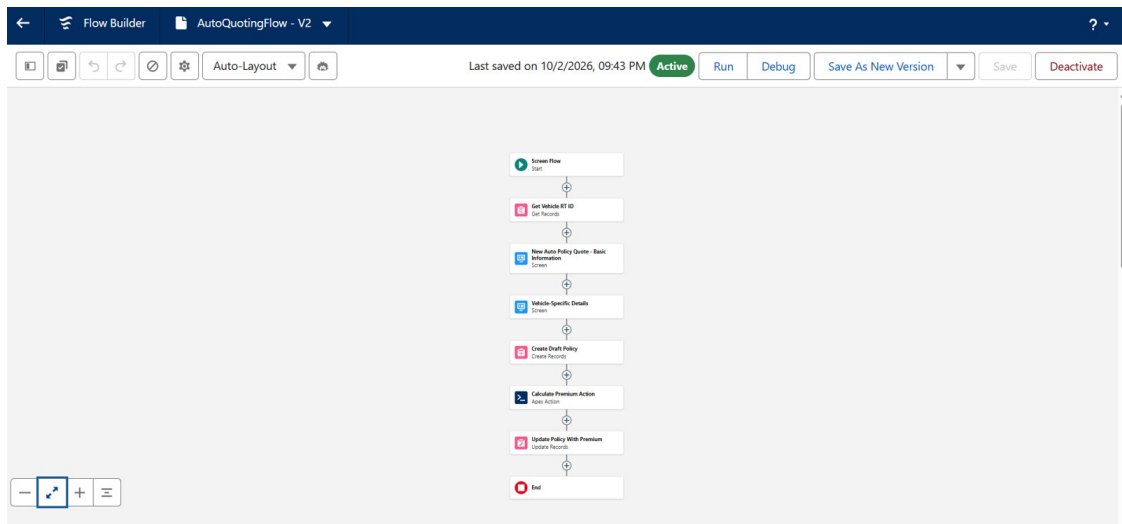


Figure 16: AutoQuotingFlow canvas (Active, Version 2)

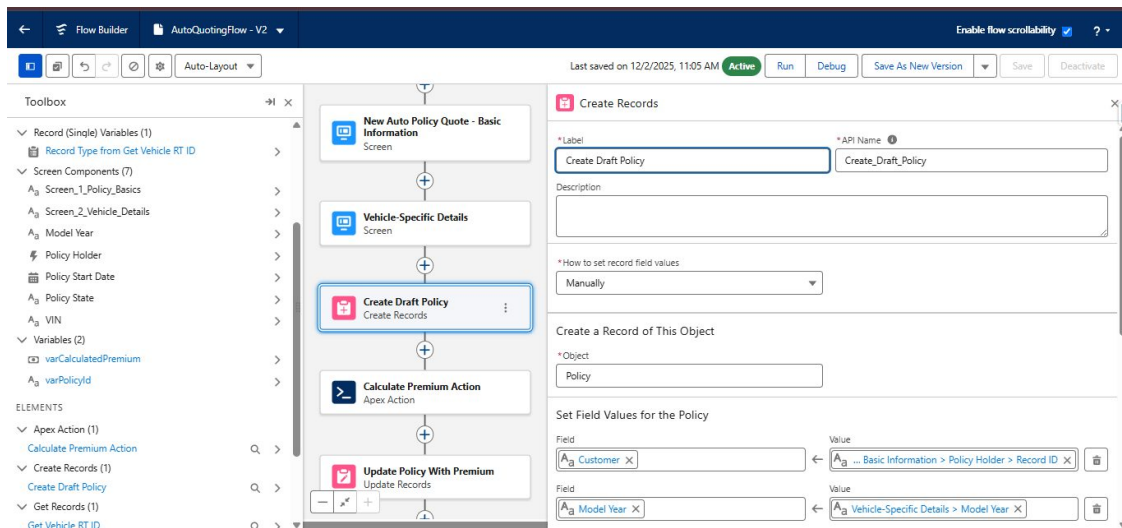


Figure 17: Create Draft Policy element

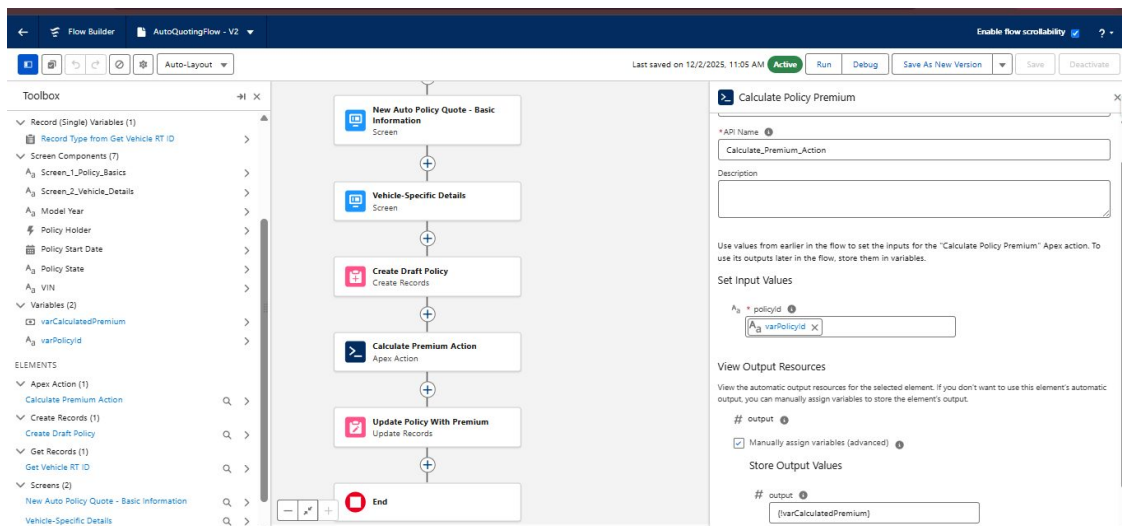


Figure 18: Calculate Premium Action element

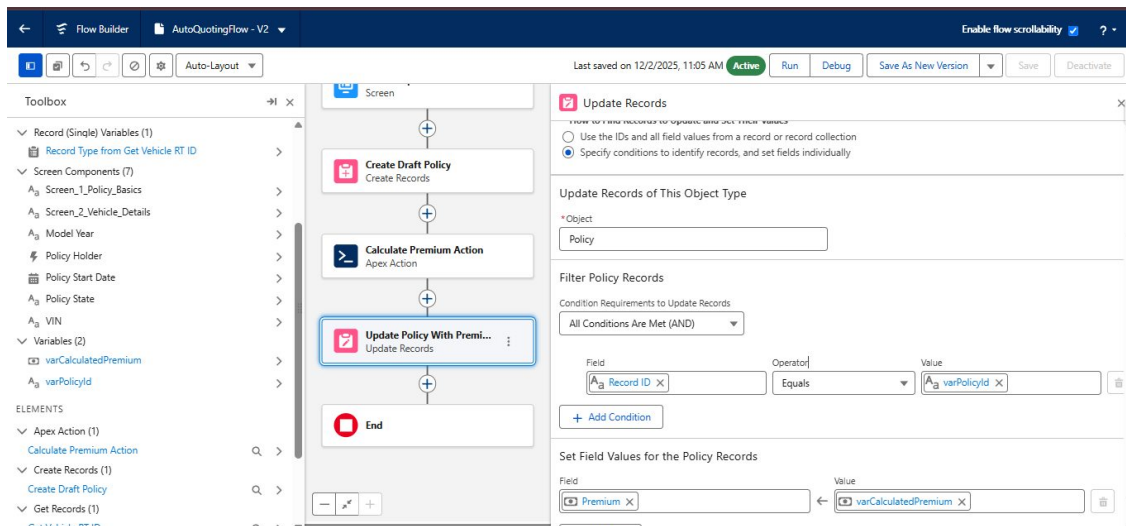


Figure 19: Update Policy With Premium element

5.5 Premium Calculation (Apex)

PremiumCalculator is an Apex class with an invocable method, *Calculate Policy Premium*, so a Flow can call it. For each policy Id received it reads the VIN, Model Year and Policy State, and applies these rules:

Rule	Effect on premium
Base premium	1,000.00
Policy State is CA	Base multiplied by 1.15
Policy State is TX	Base multiplied by 1.05
Model Year earlier than 2018	Add 100

The full source code is in Appendix A.

5.6 Claim Routing Flow

A record-triggered Flow on Claim runs after a claim is created. It looks up the related policy, stores the policy Record Type name in varPolicyRTName and passes it to a decision element called **Route by Policy Type**.

Outcome	Condition	Action
Auto Claim Route	varPolicyRTName equals Auto	Assign to Auto Queue (sets Owner Id)
Property Claim Route	varPolicyRTName equals Property	Assign to Property Queue
Life Claim Route (default)	Anything else	Assign to Life Queue

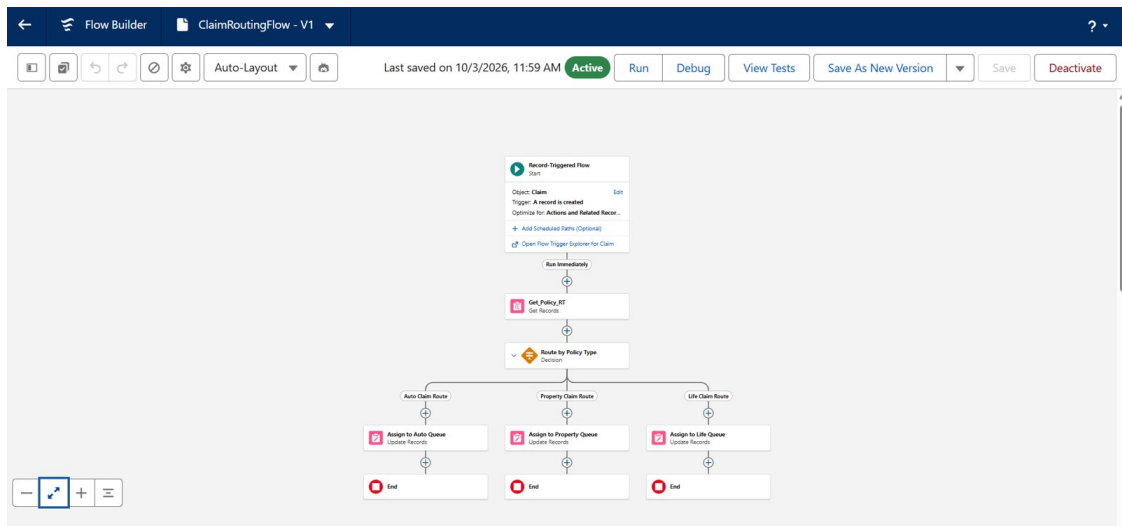


Figure 20: ClaimRoutingFlow canvas (Active)

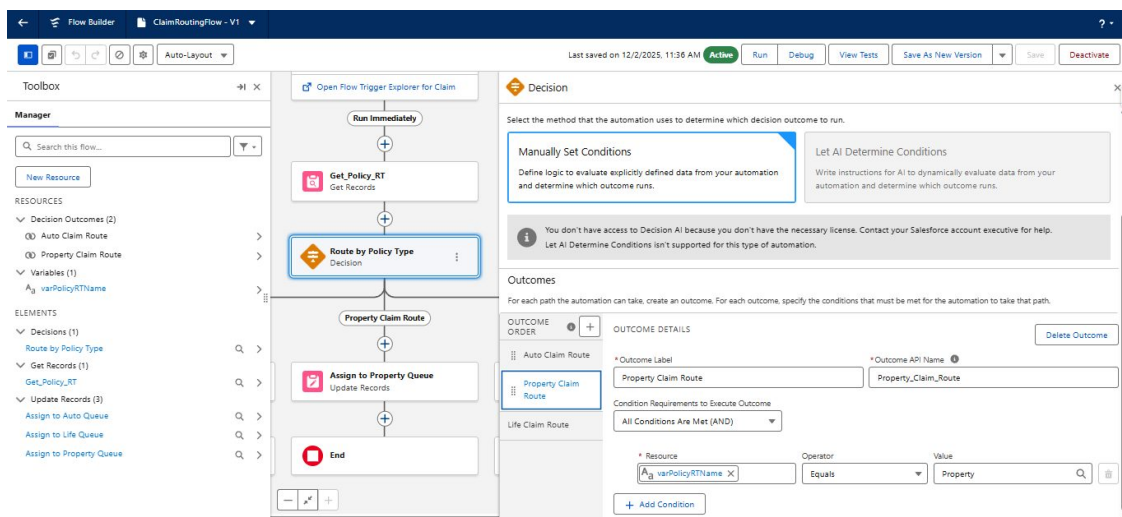


Figure 21: Decision element with the Auto and Property outcomes

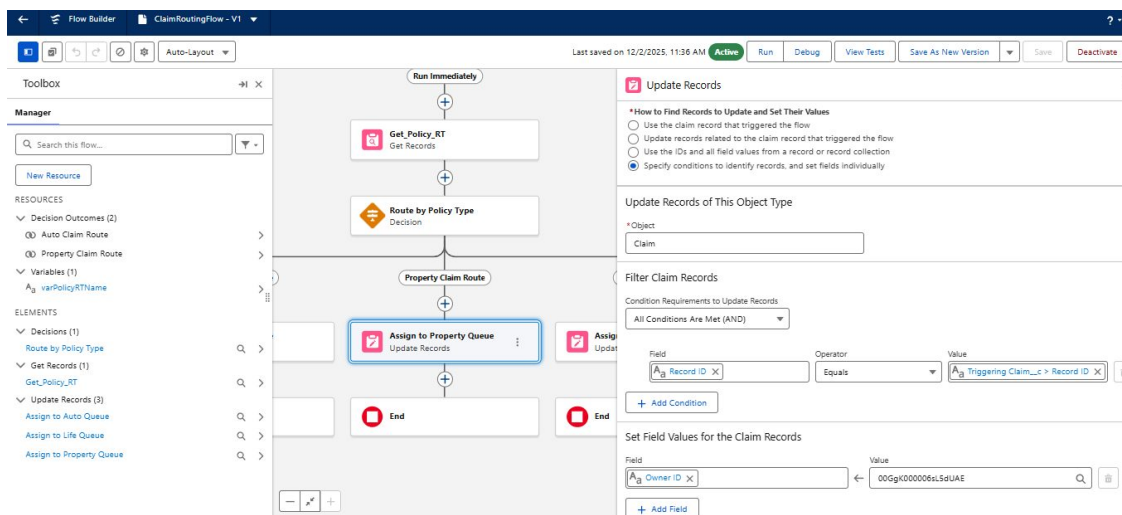


Figure 22: Update Records element that assigns the Property queue

5.7 Claims Adjuster Dashboard (LWC)

The dashboard has three parts:

- **ClaimsAdjusterController** (Apex): the cacheable method *getAssignedClaims()* queries all claims owned by the logged-in user, joins the related Policy and Contact data, and returns a list of *ClaimWrapper* objects with claim number, amount, status, policy type, policy holder and days open.
- **claimsDashboardLwc**: calls the Apex method through @wire, shows one tile per claim, and filters on the client side by All, Auto, Property or Life with a combobox.
- **claimTileLwc**: a reusable tile showing the claim number, an icon by policy type, policy holder, amount, status and days open.

The component is exposed to Lightning App pages and Home pages. All source code is in Appendix A.

5.8 Approval Process for High-Value Claims

Claims above 50,000 need extra review. The approval process **High Value Claim Approval** is defined on Claim:

Setting	Value
Process name	High Value Claim Approval (Active)
Object	Claim
Entry criteria	Claim Amount greater than 50000
Record editability while pending	Administrators only
Approval page fields	Claim Number, Owner, Claim Amount, Adjuster, Policy
Initial submitters	Claim Owner
Step 1 - Step1_Senior_Adjuster	All records enter; automatically assigned to the Senior Adjuster user; first response decides
Step 2 - Step2_Manager_Review	All records enter; automatically assigned to the Department Manager user; first response decides; on rejection perform all rejection actions
Initial submission action	Field update: Approval Status = Submitted for Approval
Final approval action	Field update: Approval Status = Approved
Final rejection action	Field update: Approval Status = Rejected

The screenshot shows the Salesforce Setup interface for configuring an approval process. The 'Approval Processes' section is selected, and the 'Specify Entry Criteria' step is active. The entry criteria are defined as follows:

Field	Operator	Value
Claim: Claim Amount	greater than	50000
--None--	--None--	
--None--	--None--	
--None--	--None--	Value 3
--None--	--None--	

The interface also includes a sidebar with navigation options like 'Data', 'Feature Settings', and 'Process Automation'. The main content area shows the 'Approval Processes' section with 'Step 2 of 6' indicated. The entry criteria are defined using a table with columns for Field, Operator, and Value.

Figure 23: Entry criteria: Claim Amount greater than 50000

Setup Home Object Manager

Q approval

Data

Mass Transfer Approval Requests

Feature Settings

Approval Settings

Process Automation

Approval Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Step1 Senior Adjuster

Step 1 of 3

Step 1. Enter Name and Description

Enter a name, description, and step number for your new approval step.

Enter Name and Description

Approval Process Name: High Value Claim Approval

Name: Step1 Senior Adjuster

Unique Name: Step1_Senior_Adjuster

Description:

Save Next Cancel

Figure 24: Step 1: Senior Adjuster

Setup Home Object Manager

Q approval

Data

Mass Transfer Approval Requests

Feature Settings

Approval Settings

Process Automation

Approval Processes

Didn't find what you're looking for? Try using Global Search.

Approval Processes

Select Approver

Let the submitter choose the approver manually.

Automatically assign to queue.

Automatically assign to approver(s).

User: Claims Manager

Add Row Remove Row

When multiple approvers are selected:

Approve or reject based on the FIRST response.

Require UNANIMOUS approval from all selected approvers.

The approver's delegate may also approve this request.

Reject Behavior

What should happen if the approver rejects this request?

Perform all rejection actions for this step AND all final rejection actions. (Final Rejection)

Perform ONLY the rejection actions for this step and send the approval request back to the most recent approver. (Go Back 1 Step)

Previous Save Cancel

Figure 25: Step 2: Manager Review

Submission Automation Flow

A record-triggered Flow named **Submission Automation Flow** runs when a Claim is created or updated, and the Claim Amount is greater than 50000 and Approval Status equals New. Its action *Submit Claim to Approval* sends the record to High Value Claim Approval.

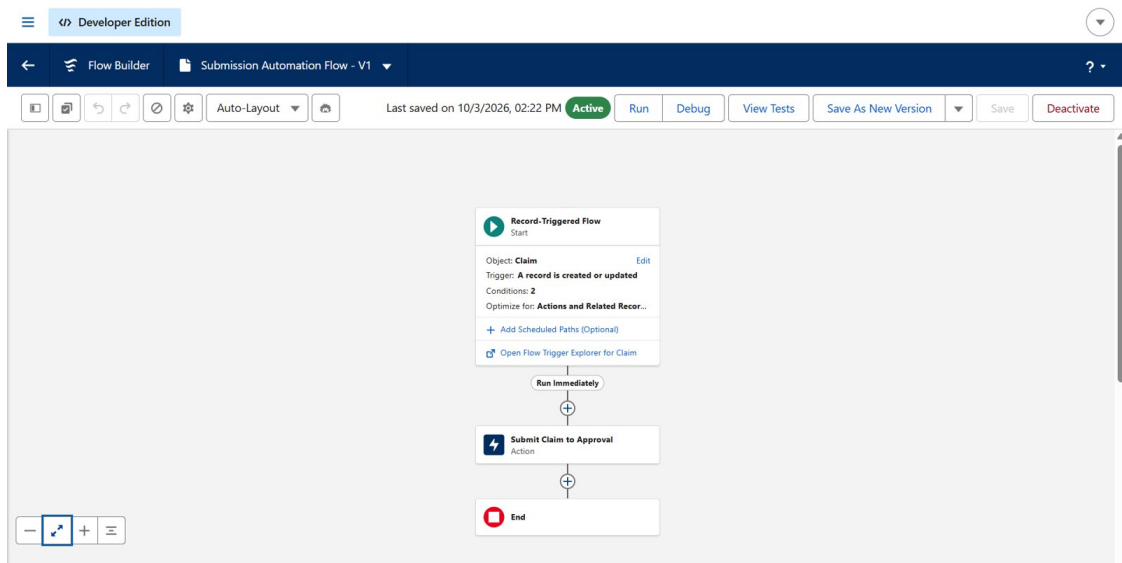


Figure 26: Submission Automation Flow (Active)

Claim Approver Screen Flow and Quick Action

Approvers need a quick way to decide. **Claim Approver Screen Flow** takes the record Id, loads the claim (Get Claim Details), shows Claim ID, Amount and Policy Type, then asks for an Approval Decision (Approve or Reject) and Approver Comments. A decision element then runs the Approve Claim or Reject Claim action with the comments. The flow is placed on the Claim page as the quick action **Approve/Reject Claim**.

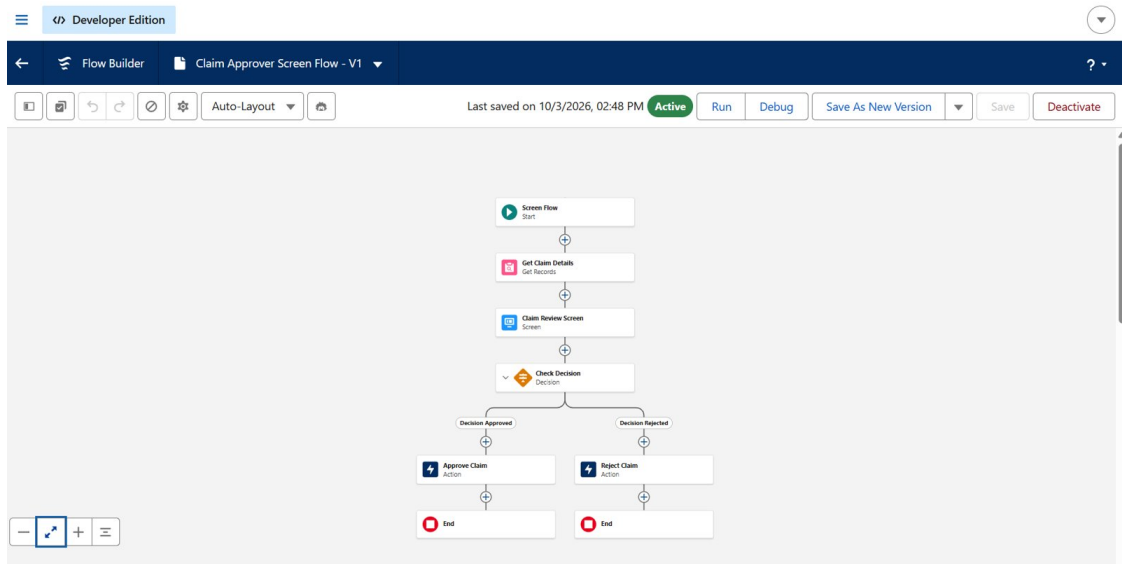


Figure 27: Claim Approver Screen Flow canvas with Approve and Reject paths

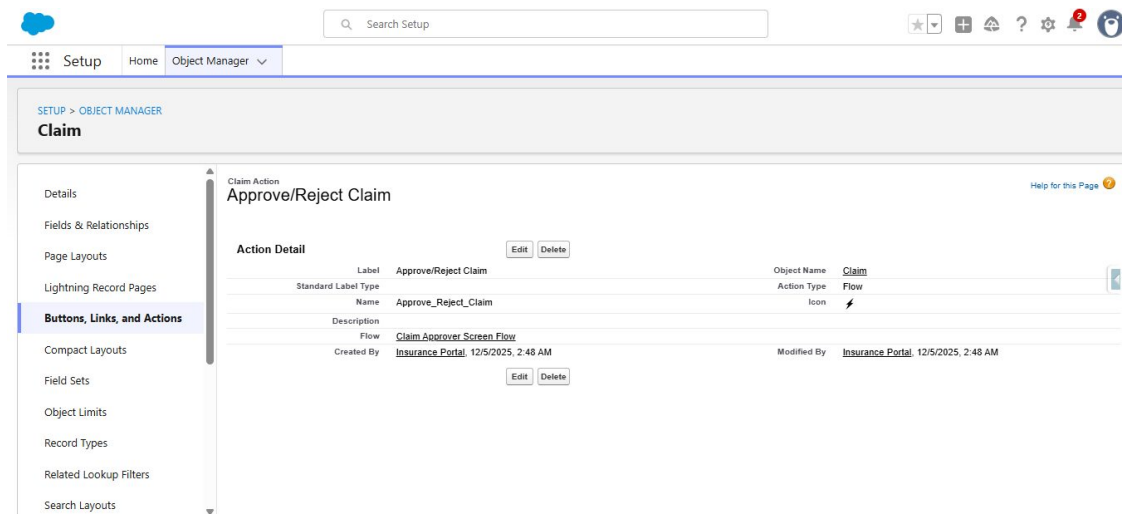


Figure 28: Quick action Approve/Reject Claim on the Claim object

5.9 Security

Access is controlled with a flow, sharing rules and Permission Sets so that each user sees and does only what the job needs.

State-based access

Claim does not hold a state, so the record-triggered Flow **Claim Policy Holder State Update** reads the Policy State from the policies and copies it into the claim's Policy Account Holder State field. Two Claim sharing rules then grant Read/Write access: *Share Records based on Policy State CA* (state equals CA) and *Share Records based on Policy State* (state equals TX).

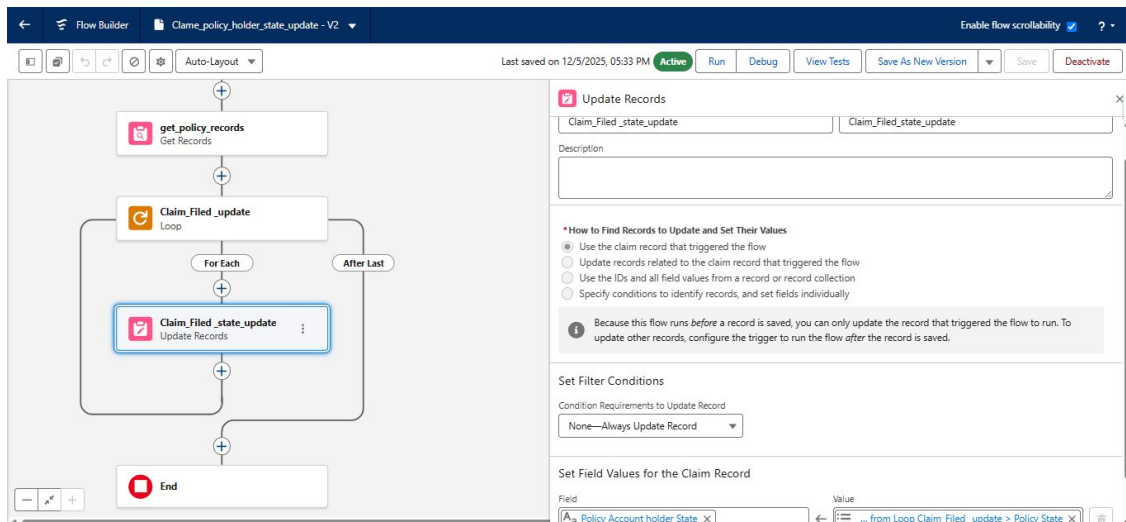


Figure 29: Flow that copies the policy state onto the claim

The Sharing Settings interface shows a rule named 'Share_Records_based_on_I'. The criteria are 'Policy Account holder State' equals 'CA'. The access level is 'Read/Write'. The rule is created by 'Insurance Portal' on 12/7/2025, 8:34 PM.

Sharing Settings Configuration:

- Label: Share Records based on Pol
- Rule Name: Share_Records_based_on_I
- Description:
- Step 1: Select your rule type
- Criteria:

Field	Operator	Value
Policy Account holder State	equals	CA
--None--	--None--	
--None--	--None--	
--None--	--None--	
--None--	--None--	
- Additional Options:
 - Include records owned by users who can't have an assigned role
- Share with: Group: Claim Adjusters- California
- Access Level: Read/Write
- Created By: Insurance Portal 12/7/2025, 8:34 PM
- Modified By: Insurance Portal 12/7/2025, 8:34 PM

Figure 30: Sharing rule for state CA

The Sharing Settings interface shows a rule named 'Share_Records_based_on_I'. The criteria are 'Policy Account holder State' equals 'TX'. The access level is 'Read/Write'. The rule is created by 'Insurance Portal' on 12/7/2025, 8:33 PM.

Sharing Settings Configuration:

- Label: Share Records based on Pol
- Rule Name: Share_Records_based_on_I
- Description:
- Step 1: Select your rule type
- Criteria:

Field	Operator	Value
Policy Account holder State	equals	TX
--None--	--None--	
--None--	--None--	
--None--	--None--	
--None--	--None--	
- Additional Options:
 - Include records owned by users who can't have an assigned role
- Share with: Group: Claim Adjusters- Texas
- Access Level: Read/Write
- Created By: Insurance Portal 12/7/2025, 8:33 PM
- Modified By: Insurance Portal 12/7/2025, 8:33 PM

Figure 31: Sharing rule for state TX

Permission Sets

Permission Set	Object access	Other permissions
Insurance Agent Access	Policy: Create, Read. Contact: Read, Edit. Claim: Read only.	Flow User enabled; PremiumCalculator Apex class enabled.
Claims Adjuster Access	Policy: Read only. Claim: Read, Edit.	ClaimsAdjusterController Apex class enabled.
Claims Manager Access	Policy: Read only. Contact: Read, Edit. Claim: Read, Edit, Delete.	Run Reports and Manage Approvals enabled.

The screenshot shows the Salesforce Setup interface for creating a new permission set. The left sidebar shows the navigation menu with 'Permission Sets' selected. The main content area is titled 'Permission Sets' and contains a form for 'Enter permission set information'. The form fields are: Label (Insurance Agent Access), API Name (Insurance_Agent_Access), Description (empty), and Session Activation Required (unchecked). Below the form, there is a section for 'Select the type of users who will use this permission set' with a 'License' dropdown set to '--None--'.

Figure 32: Creating the Insurance Agent Access Permission Set

5.10 Apex Test Class

ClaimsAdjusterControllerTest creates a test user, a Contact, an Auto policy and an Accident claim all owned by that user. It then runs `getAssignedClaims()` as the test user and asserts that the claim number, status (New), amount (5000) and policy holder name are returned and that the policy type is not Unknown. The source is in Appendix A.

The screenshot shows the Salesforce Developer Console with the Apex test class `ClaimsAdjusterControllerTest` open in the editor. The test class code is as follows:

```
@isTest
private class ClaimsAdjusterControllerTest {
    @testSetup
    static void setupTestData() {
        // Create a real active test user
        Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User' LIMIT 1];
        User u = new User(
            FirstName = 'Test',
            LastName = 'User',
            Alias = 'tuser',
            Username = 'testuser1234343@example.com.test',
            Email = 'testuser123@example.com',
        );
    }
}
```

The bottom pane shows the test results for the `ClaimsAdjusterControllerTest` class. The test results table is as follows:

Status	Test Run	Enqueued Time	Duration	Failures	Total
✗	TestRun @ 11:58:17 am			1	1
✗	TestRun @ 12:01:35 pm			1	1
✗	TestRun @ 12:07:16 pm			1	1
✗	TestRun @ 12:14:03 pm			1	1
✗	TestRun @ 12:22:36 pm			1	1
✗	TestRun @ 12:27:50 pm			1	1
✓	TestRun @ 12:29:39 pm			0	1

The right pane shows the 'Overall Code Coverage' for the test class. The coverage is 48% for the `ClaimsAdjusterController` class, with 20/23 lines covered. The `PremiumCalculator` class has 0% coverage with 0/18 lines covered.

Figure 33: Running ClaimsAdjusterControllerTest in the Developer Console

6. Testing

Each feature was tested from the Lightning interface. The table lists the test cases with the behaviour expected from the design.

No.	Test case	Expected result	Evidence
1	Save an Auto policy with a VIN of fewer or more than 17 characters	Save is blocked with the VIN error message	Tested in org: save blocked, VIN error shown
2	Save an Auto policy with a 17-character VIN	Policy saves	Tested in org: policy saved
3	Run AutoQuotingFlow with valid inputs	Draft policy is created and Premium is filled in	Tested in org: flow ran, Premium filled
4	Premium for a CA policy, model year 2020	1,150.00	Tested in org: Premium 1,150.00
5	Premium for a TX policy, model year 2015	1,150.00 (1,000 x 1.05 + 100)	Tested in org: Premium 1,150.00
6	Create a claim on an Auto, Property and Life policy	Owner becomes the Auto, Property or Life queue	Tested in org: Owner set to correct queue
7	Create a claim with amount above 50,000	Claim goes to approval; status Submitted for Approval	Tested in org: status Submitted for Approval
8	Approve or reject from the Approve/Reject Claim action	Status becomes Approved or Rejected	Tested in org: status Approved / Rejected
9	Open the Claims Dashboard and use the filter	Only the user's claims show; filter works	Tested in org: own claims shown, filter works
10	Run ClaimsAdjusterControllerTest	Test passes with coverage on the controller	Figure 33
11	CA and TX users open claims	Users see claims from their state	Tested in org: state-wise access checked

7. Challenges Faced and Fixes

The main design problems and how each was solved are described below.

7.1 One object for three insurance lines

Vehicle, Property and Life policies need different fields and different validations. Creating three separate objects would have duplicated automation and reports. We kept a single Policy object and used Record Types and Field Sets so that each product shows only its own fields, and the VIN rule applies only to the Auto Record Type.

7.2 Routing claims by policy type

A claim does not know the type of its policy. The routing Flow therefore reads the policy first and stores its Record Type name in a variable, then a decision element chooses the queue. The queue Ids are entered in the Update Records elements, so they must be updated if a queue is recreated.

7.3 Showing the state for sharing rules

Sharing rules can only use fields on the object being shared. Because the state lives on the Policy, we added a Flow that copies the Policy State to the claim, and then built the CA and TX sharing rules on that field.

7.4 Calling Apex from a Flow

The Flow can only call an Apex method that is invocable. PremiumCalculator therefore uses an invocable method with a wrapper class that takes the policy Id, and the Flow saves the draft policy first so that the Id exists before the premium is calculated.

8. Limitations and Future Scope

Current limitations of the solution:

- The premium rule is a simple model: it uses a base amount, the states CA and TX, and a model year check. Other states, vehicle values and Property or Life rating are not yet covered.
- The Auto Quoting Flow handles the Auto line only; Property and Life quotes still use manual policy creation.
- Queue Ids and user names in the approval steps are set directly in the configuration, so they need maintenance when people or queues change.
- The dashboard shows Days Open from the Date of Loss and does not yet stop counting when a claim is closed.
- Test coverage is written for ClaimsAdjusterController; PremiumCalculator and the Flows need their own tests before a production deployment.
- Sharing rules exist for the states CA and TX only.

Ideas for future work:

- Integrate with external vehicle and property databases for real-time quote verification.
- Add more insurance products beyond Vehicle, Property and Life.
- Add automated claim assessment and escalation based on claim characteristics.
- Build richer reports and dashboards for policy, premium and claims analysis.
- Offer a customer portal (Experience Cloud) to track policy and claim status, plus mobile access for agents and adjusters.
- Explore AI assistance for agents and adjusters.

9. Conclusion

The Multi-Line Insurance Policy and Claims Management System brings policy quoting, premium calculation, claim routing, claim approval and adjuster reporting into one Salesforce application. Record Types and Field Sets keep three insurance lines in one data model; the validation rule and Screen Flow protect data quality; Apex calculates premiums; Flows route claims and start approvals; the LWC dashboard gives adjusters a focused view; and sharing rules with Permission Sets control access. Building it gave our team practical experience with objects, Flows, Apex, Lightning Web Components, approval processes and security, and the same design can be extended with more products, integrations and a customer portal.

Appendix A: Apex and LWC Source Code

PremiumCalculator

```
public class PremiumCalculator {
    public class PremiumInputWrapper {
        @InvocableVariable(required=true)
        public Id policyId;
    }

    @InvocableMethod(label='Calculate Policy Premium'
        description='Calculates premium based on policy fields (VIN, Model Year, State).')
    public static List<Decimal> calculatePremium(List<PremiumInputWrapper> inputs) {
        List<Decimal> premiumResults = new List<Decimal>();
        Set<Id> policyIds = new Set<Id>();
        for (PremiumInputWrapper input : inputs) {
            if (input.policyId != null) {
                policyIds.add(input.policyId);
            }
        }
        List<Policy__c> policiesToRate = [
            SELECT Id, VIN__c, Model_Year__c, Policy_State__c
            FROM Policy__c
            WHERE Id IN :policyIds
        ];
        for (Policy__c policy : policiesToRate) {
            Decimal basePremium = 1000.00;
            if (policy.Policy_State__c == 'CA') {
                basePremium *= 1.15;
            } else if (policy.Policy_State__c == 'TX') {
                basePremium *= 1.05;
            }
            if (policy.Model_Year__c != null && Integer.valueOf(policy.Model_Year__c) < 2018) {
                basePremium += 100;
            }
            premiumResults.add(basePremium);
        }
        return premiumResults;
    }
}
```

ClaimsAdjusterController

```
public class ClaimsAdjusterController {
    public class ClaimWrapper {
        @AuraEnabled public Id claimId;
        @AuraEnabled public String claimNumber;
        @AuraEnabled public Decimal claimAmount;
        @AuraEnabled public String status;
        @AuraEnabled public String policyType;
        @AuraEnabled public String policyHolderName;
        @AuraEnabled public Integer daysOpen;

        public ClaimWrapper(Claim__c c, Policy__c p, Contact customerRec) {
            this.claimId = c.Id;
            this.claimNumber = c.Name;
            this.claimAmount = c.Claim_Amount__c;
            this.status = c.Approval_Status__c;
            Policy__c policyRecord = (Policy__c)p;
            if (policyRecord != null && policyRecord.RecordType != null) {
                this.policyType = policyRecord.RecordType.Name;
            } else {
                this.policyType = 'Unknown';
            }
            this.policyHolderName = (customerRec != null) ? customerRec.Name : '';
            this.daysOpen = (c.Date_of_Loss__c != null)
                ? (c.Date_of_Loss__c.date().daysBetween(Date.today()))
                : 0;
        }
    }
    @AuraEnabled(cacheable=true)
```

```

public static List<ClaimWrapper> getAssignedClaims() {
    List<ClaimWrapper> results = new List<ClaimWrapper>();
    List<Claim__c> claims = [
        SELECT Id, Name, Claim_Amount__c, Approval_Status__c, Date_of_Loss__c,
            Policy__r.Name, Policy__r.RecordType.DeveloperName,
            Policy__r.Customer__r.FirstName, Policy__r.Customer__r.LastName
        FROM Claim__c
        WHERE OwnerId = :UserInfo.getUserId()
        ORDER BY Date_of_Loss__c DESC
    ];
    for (Claim__c claim : claims) {
        Policy__c policy = claim.Policy__r;
        Contact customer = (policy != null && policy.Customer__r != null)
            ? (Contact)policy.Customer__r
            : new Contact();
        results.add(new ClaimWrapper(claim, policy, customer));
    }
    return results;
}
}

```

ClaimsAdjusterControllerTest

```

@isTest
private class ClaimsAdjusterControllerTest {
    @testSetup
    static void setupTestData() {
        Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User' LIMIT 1];
        User u = new User(
            FirstName = 'Test', LastName = 'User', Alias = 'tuser',
            Username = 'testuser1234343@example.com.test', Email = 'testuser123@example.com',
            TimeZoneSidKey = 'America/Los_Angeles', LocaleSidKey = 'en_US',
            EmailEncodingKey = 'UTF-8', LanguageLocaleKey = 'en_US', ProfileId = p.Id
        );
        insert u;
        Id policyRT = Schema.SObjectType.Policy__c.getRecordTypeInfoByDeveloperName()
            .get('Auto').getRecordTypeId();
        Id claimRT = Schema.SObjectType.Claim__c.getRecordTypeInfoByDeveloperName()
            .get('Accident').getRecordTypeId();
        Contact con = new Contact(FirstName = 'John', LastName = 'Doe', OwnerId = u.Id);
        insert con;
        Policy__c policy = new Policy__c(Customer__c = con.Id, RecordTypeId = policyRT,
            OwnerId = u.Id, VIN__c = '1A903284375893532');
        insert policy;
        Claim__c claim = new Claim__c(Policy__c = policy.Id, RecordTypeId = claimRT,
            Claim_Amount__c = 5000, Date_of_Loss__c = Date.today().addDays(-10), OwnerId = u.Id);
        insert claim;
    }

    @isTest
    static void testGetAssignedClaims() {
        User u = [SELECT Id FROM User WHERE Alias = 'tuser' LIMIT 1];
        System.runAs(u) {
            Test.startTest();
            List<ClaimsAdjusterController.ClaimWrapper> results =
                ClaimsAdjusterController.getAssignedClaims();
            Test.stopTest();
            ClaimsAdjusterController.ClaimWrapper wrap = results[0];
            System.assertNotEquals(null, wrap.claimId);
            System.assertNotEquals(null, wrap.claimNumber);
            System.assertEquals('New', wrap.status);
            System.assertEquals(5000, wrap.claimAmount);
            System.assert(wrap.policyHolderName.contains('John'));
            System.assertNotEquals('Unknown', wrap.policyType);
        }
    }
}

```

claimsDashboardLwc - HTML

```

<template>
    <lightning-card title="My Assigned Claims Dashboard" icon-name="standard:claim">

```

```

<div class="slds-m-around_medium">
  <lightning-combobox name="policyFilter" label="Filter by Policy Type" value="All"
    options={policyTypeOptions} onchange={handleFilterChange}
    class="slds-m-bottom_medium"></lightning-combobox>
  <template if:true={visibleClaims}>
    <div class="slds-grid slds-wrap slds-grid_pull-padded">
      <template for:each={visibleClaims} for:item="claim">
        <div key={claim.claimId} class="slds-col slds-size_1-of-1
          slds-medium-size_1-of-2 slds-large-size_1-of-3 slds-p-horizontal_small">
          <c-claim-tile-lwc claim-data={claim}></c-claim-tile-lwc>
        </div>
      </template>
    </div>
    <p class="slds-m-top_medium slds-text-heading_small">
      Total Claims Assigned: {visibleClaims.length}</p>
  </template>
  <template if:false={visibleClaims}>
    <p>No claims currently assigned to you.</p>
  </template>
</div>
</lightning-card>
</template>

```

claimsDashboardLwc - JavaScript

```

import { LightningElement, wire, track } from 'lwc';
import getAssignedClaims from '@salesforce/apex/ClaimsAdjusterController.getAssignedClaims';

export default class ClaimsDashboardLwc extends LightningElement {
  @track allClaims = [];
  @track visibleClaims = [];
  error;

  @wire(getAssignedClaims)
  wiredClaims({ error, data }) {
    if (data) {
      this.allClaims = data;
      this.visibleClaims = data;
      this.error = undefined;
    } else if (error) {
      this.error = error;
      this.allClaims = undefined;
      this.visibleClaims = undefined;
      console.error('Error retrieving claims:', JSON.stringify(error));
    }
  }

  handleFilterChange(event) {
    const selectedPolicyType = event.target.value;
    if (selectedPolicyType === 'All') {
      this.visibleClaims = this.allClaims;
    } else {
      this.visibleClaims = this.allClaims.filter(
        claim => claim.policyType === selectedPolicyType);
    }
  }

  get policyTypeOptions() {
    return [
      { label: 'All Policy Types', value: 'All' },
      { label: 'Auto', value: 'Auto' },
      { label: 'Property', value: 'Property' },
      { label: 'Life', value: 'Life' }
    ];
  }
}

```

claimsDashboardLwc - meta XML

```

<?xml version="1.0"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>63.0</apiVersion>

```

```

<isExposed>true</isExposed>
<targets>
  <target>lightning__AppPage</target>
  <target>lightning__HomePage</target>
</targets>
</LightningComponentBundle>

```

claimTileLwc - HTML

```

<template>
  <lightning-layout-item size="12" padding="around-small">
    <div class="tile slds-box slds-p-around_medium slds-theme_default">
      <header class="slds-media slds-media_center slds-m-bottom_small">
        <div class="slds-media__figure">
          <lightning-icon icon-name={policyIcon} alternative-text={claimData.policyType}
            size="small"></lightning-icon>
        </div>
        <div class="slds-media__body slds-m-left_small">
          <h2 class="slds-text-heading_medium slds-m-bottom_none">
            Claim: {claimData.claimNumber}</h2>
            <p class="slds-text-body_small">Policy Type: <strong>{claimData.policyType}</strong></p>
          </div>
        </header>
        <div class="slds-border_top slds-p-top_small">
          <p>Policy Holder: <strong>{claimData.policyHolderName}</strong></p>
          <lightning-formatted-number value={claimData.claimAmount} format-style="currency"
            currency-code="USD" label="Claim Amount"></lightning-formatted-number>
          <p>Status: {claimData.status}</p>
          <p class="slds-text-body_regular">Days Open:
            <strong class="slds-text-color_error">{claimData.daysOpen}</strong></p>
        </div>
      </div>
    </lightning-layout-item>
  </template>

```

claimTileLwc - JavaScript

```

import { LightningElement, api } from 'lwc';

export default class ClaimTileLwc extends LightningElement {
  @api claimData;

  get policyIcon() {
    if (this.claimData && this.claimData.policyType) {
      const type = this.claimData.policyType.toLowerCase();
      if (type === 'auto') return 'utility:truck';
      if (type === 'property') return 'utility:home';
      if (type === 'life') return 'utility:connected_apps';
    }
    return 'utility:help';
  }
}

```